Adversarial Test Report

Deliverable 3, Campus Ride-Pooling and Split-Fare System

Prepared by Mainak Sarkar Jira CS455-113, CS455-114 Run of 2026-10-29

Prithviraj Ghosh (230794) Mainak Sarkar (230619) Khushi Ranawat (230562) Mahir Jain (230618) Chaitanya Nitawe (230315)

Result. 41 adversarial cases in 4 files try to break the app through its real API routes against a real PostgreSQL database. After the fix below, all 41 pass. The suite found one real weakness (an idempotency key replaying the wrong request), fixed in PR #87 with regression tests. It also confirmed that every race it staged is serialized by the database, and it records one observation for the security report.

1. Purpose and method

Ordinary tests show that the app does what it should when used as intended. Adversarial tests use it as it should *not* be used: at the wrong moment, with the wrong input, twice, out of order, or several times at once. They run on the integration harness (see the integration test report): the real route handlers, the real SQL, a real database emptied before every case.

Pass criterion. Every case must end in one of two ways: a clean refusal (a 4xx with a reason code, nothing stored), or a consistent state (the rows obey the rules, for example never a seat without an accepted request). A 500, a partial write or a misleading success fails the case.

Boundaries with the other D3 reports. Access control (one user acting on another's records), cross-site requests and secrets are the security report's subject; the 50-request last-seat experiment (NFR-RD-05) is the concurrency report's. This suite keeps a small version of each race as a regression test, run on every pull request.

2. Attack categories

Category	What is attacked, and why
Time boundaries	Every rule that depends on the clock, a moment either side of its boundary: the lock an hour before departure (P-06), the five-minute no-show grace period (P-19), the seven-day complaint window (FR-RD-13.1), the 72-hour rating window (P-16), the end of the departure window. Off-by-one errors (`<` against `<=`) live here.
Input	Bodies that are not JSON objects, wrong types that a lenient parser might coerce (`"600"`, 600.5, a timestamp number, month 13), oversized strings up to 1 MB, malformed, injection-looking and path-like identifiers, SQL and markup as text.
Replays	The same request sent again: a double-clicked accept, and an idempotency key sent again with *different* details.
Order	Actions that are valid in one state sent in another: requests and edits on a cancelled ride, leaving or checking in after pickup started, pickup actions and SOS after completion, acting again after leaving.
Races	Two to five requests at the same moment, each on its own database connection so they really overlap: two accepts for the last seat, accept against withdraw, a five-fold double-click, triple lock, five SOS taps, completion twice, the same offer twice; and the per-address rate limit.

3. Results

Table 1. Adversarial test files, from the recorded run (`test-run/integration-results.json`).

File (under `apps/web/test/integration/`)	Cases	Passed	Time (s)
`adversarial/input.int.test.ts`	15	15	3.8
`adversarial/races.int.test.ts`	8	8	3.3
`adversarial/replays-and-state.int.test.ts`	7	7	2.5
`adversarial/time-boundaries.int.test.ts`	11	11	3.7
Total	**41**	**41**	

Every case is listed in the appendix. The race file was also run five times in a row on its own, passing each time, to check that its results do not depend on timing.

4. Defect found: an idempotency key replayed the wrong request

Case	`replays-and-state`: "does not answer a request for one ride with a request on another", and "refuses a different pickup sent with an idempotency key already used".
Steps	A rider requests a seat on ride A with key K (201). They then request a seat on ride B, or on ride A with another pickup point, with the same key K.
Expected	A refusal: K belongs to a different request (FR-RD-06.9 makes a key replay *the same* request).
Actual	200 `{kind: "existing", requestId: <ride A's request>}`. The page says "This request was already submitted"; no request exists on ride B, or the new pickup point is silently dropped. The UI keeps its key across failed tries and can stay mounted across client-side navigation, so a real rider could hit this.
Cause	`submitJoinRequest()` looked the key up and returned its request without comparing what that request was for.
Fix	PR #87: a key replays only the request it was first sent with (same ride and pickup point); anything else gets 409 `idempotency_key_reused` and nothing is created. The request form drops its key on that answer.
Regression	`bugs/idempotency-key-reuse` (5 tests; the two mismatch cases fail without the fix) and 2 unit cases in `join-requests.test.ts`. Jira Bug, labels `d3-defect`, `found-by-adversarial`.
Review	Asked on PR #87 whether two first uses of a new key could both pass the lookup. They cannot (next section); two tests added after review pin it: four simultaneous first uses for two rides, in five rounds, and a direct second insert refused by the key table's primary key.

5. What held, and why

The other cases passed on the first run. For the races the reason is worth recording, since it is what the concurrency report measures at scale:

- **Last seat, accept against withdraw, double accept.** Each decision locks the rider's `users` row and the ride row (`FOR UPDATE` / `FOR NO KEY UPDATE`) and re-reads the seats inside that lock, so the second decision sees the first one's result: `ride_full` or `request_not_active`, never an extra seat.
- **Double-clicked submit.** The rider's row lock serializes the five requests; the first stores the key, the other four find it and replay the same request. When the first uses of a new key are for *different* rides, the same lock lets one create its request; the rest replay it or get 409. With the lock removed, that test fails: the race reaches the key table's primary key, which still refuses a second claim but answers 500.
- **SOS taps.** A partial unique index allows one active alert per person per ride, and the insert uses `ON CONFLICT DO NOTHING`, so taps cannot create a second alert even without a lock.
- **Completion twice, triple lock.** Trip steps lock the ride row and treat "already there" as done; the final shares are written once and still add up to the fare.
- **Input.** Route IDs are checked as UUIDs before any query; bodies go through one tolerant reader (`readJson`) and typed parsers that refuse rather than coerce; SQL is always parameterized, so hostile text is stored as text.

6. Observation for the security report

The per-address rate limit keys on the first value of the `X-Forwarded-For` header (`clientIp()` in `src/lib/auth.ts`). The suite relies on this to give each call its own address, and the rate-limit case shows that a new address starts a new count. Unless the server sits behind a proxy that replaces that header, a client can choose its own address and avoid the limit. Whether the deployment's proxy overwrites the header should be checked in the security report; it is not counted as a defect here.

7. Limitations

- The races use two to five requests; heavier contention is the concurrency report's experiment.

- Time boundaries are tested between six seconds and a minute either side, by moving data rather than the clock (see the integration report); exact-millisecond behaviour is covered by the unit tests of the rule functions.
- Access control and cross-user abuse are deliberately left to the security report.

A. Every adversarial case

File	Case (describe block: test)	Result
input	malformed bodies: offering a ride with broken JSON is a 400, not a crash	Pass
input	malformed bodies: offering a ride with a JSON array is a 400, not a crash	Pass
input	malformed bodies: offering a ride with a JSON string is a 400, not a crash	Pass
input	malformed bodies: offering a ride with null is a 400, not a crash	Pass
input	malformed bodies: offering a ride with an empty body is a 400, not a crash	Pass
input	malformed bodies: refuses wrong types instead of coercing them	Pass
input	malformed bodies: refuses an edit with a string capacity or a non-boolean confirmation	Pass
input	oversized input: refuses an idempotency key or pickup ID beyond its limit	Pass
input	oversized input: refuses a complaint description over 2000 characters, and a megabyte of it	Pass
input	identifiers: treats the ride ID "not-a-uuid" as not found without querying for it	Pass
input	identifiers: treats the ride ID "' OR '1'='1" as not found without querying for it	Pass
input	identifiers: treats the ride ID "00000000-0000-0000-0000-000000000000; DROP TABLE rides;--" as not found without querying for it	Pass
input	identifiers: treats the ride ID "../../etc/passwd" as not found without querying for it	Pass
input	identifiers: answers a well-formed but unknown ride ID with the same 404	Pass
input	hostile text: stores SQL and markup in a complaint as plain text	Pass
races	requests at the same moment: gives the last seat to exactly one of two accepted riders	Pass
races	requests at the same moment: never leaves a seat without an accepted request when accept and withdraw race	Pass
races	requests at the same moment: creates one request for a double-clicked submit with one key	Pass
races	requests at the same moment: locks once when the owner double-clicks Lock	Pass
races	requests at the same moment: raises one SOS for five quick taps	Pass
races	requests at the same moment: writes the final shares once when completion is sent twice at once	Pass
races	requests at the same moment: offers one ride when the same offer is sent twice at once	Pass
races	the per-address rate limit: turns away the 21st ride offer from one address in 15 minutes	Pass
replays-and-state	replayed requests: refuses a different pickup sent with an idempotency key already used (bug 2, #87)	Pass
replays-and-state	replayed requests: does not answer a request for one ride with a request on another (bug 2, #87)	Pass
replays-and-state	replayed requests: accepts a request once, however often the owner clicks	Pass
replays-and-state	actions out of order: refuses requests, decisions and edits on a cancelled ride	Pass
replays-and-state	actions out of order: refuses leaving, checking in and new requests once pickup has started	Pass
replays-and-state	actions out of order: refuses pickup actions and SOS after completion	Pass
replays-and-state	actions out of order: refuses a rider who left acting on the ride again	Pass
time-boundaries	the lock, an hour before departure (P-06): departing in 60.5 minutes: a seat request answers 201	Pass
time-boundaries	the lock, an hour before departure (P-06): departing in 59.5 minutes: a seat request answers 409	Pass
time-boundaries	the lock, an hour before departure (P-06): drops a ride from search results at the same minute	Pass
time-boundaries	the no-show grace period, five minutes after the agreed pickup (P-19): 4.9 minutes after pickup: marking a no-show answers 409	Pass
time-boundaries	the no-show grace period, five minutes after the agreed pickup (P-19): 5.1 minutes after pickup: marking a no-show answers 200	Pass
time-boundaries	the complaint window, seven days after completion (FR-RD-13.1): 7 days less a minute after completion: a complaint answers 10079	Pass
time-boundaries	the complaint window, seven days after completion (FR-RD-13.1): 7 days and a minute after completion: a complaint answers 10081	Pass

File	Case (describe block: test)	Result
`time-boundaries`	the rating window, 72 hours after completion (P-16): 72 hours less a minute after completion: rating answers 4319	Pass
`time-boundaries`	the rating window, 72 hours after completion (P-16): 72 hours and a minute after completion: rating answers 4321	Pass
`time-boundaries`	the end of the departure window: 59 minutes into a one-hour window: cancelling answers 200	Pass
`time-boundaries`	the end of the departure window: 61 minutes into a one-hour window: cancelling answers 409	Pass